Package 'etlcov'

November 10, 2026

Type Package

Title Entry-wise Transfer Learning for Correlation and Covariance Estimation

Version 1.0

Description This package provides functions for implementing entry-wise transfer learning for covariance matrix estimation (ETL-cov), which combines entry-wise correlation transfer (ETL-corr) and coordinate-wise variance transfer (ETL-var) under the correlation-variance decomposition.

License GPL-3

Encoding UTF-8

Depends R (>= 4.1.0)

Imports ks, mvtnorm, parallel, stats

NeedsCompilation no

R topics documented:

data.generate 1
fit.ETL.corr 3
fit.ETL.corr.cv 4
fit.ETL.Scor 5
ETL.var 6
ETL.var.cv 7
make.pd 9
fit.ETL.cov 10
fit.ETL.Scov 11
fit.domain.scor 12

Index **13**

`data.generate` *Generate Target and Source Data as an example*

Description

Provides an example of generating parameter matrices and data. See Details.

Usage

```
data.generate(p, K, k0, d, s0, n0, n_s_vec, seed = NULL)
```

Arguments

p	Dimension of data.
K	Number of source domains.
k_0	Number of informative sources.
d	Informative level, $d \in [0, 1]$. Larger value of d indicates that the first k_0 sources are more close to the target.
s_0	Number of largest absolute off-diagonal entries retained in each row.
n_0	Number of target observations.
n_s_vec	Source sample sizes, one per source in source order.
seed	Random seed for parameter generation and sampling. The default NULL uses and advances the current random-number stream.

Details

Start with the identity plus independent uniform $(0, 0.2)$ entries and symmetrize. Retain up to s_0 off-diagonal entries with the largest absolute values in each row and their symmetric counterparts. Add a nonnegative diagonal shift so that the target covariance Sigma0 has minimum eigenvalue at least 0.1, up to numerical precision.

Let R0 be the target correlation matrix and let D_0 contain the square roots of the target variances on its diagonal. Permute the coordinates of R0 by a shift of p/2 to obtain R_N. The reference covariance is $\Sigma_N = 2D_0 R_N D_0$. For source k, the covariance is $(1 - d)\Sigma_0 + d\Sigma_N$ when $k \leq k_0$, and Σ_N otherwise. All domains have zero mean and Gaussian samples.

Value

A list with the following components:

X0 Numeric n_0 by p target observation matrix.

XS List of K numeric source matrices; matrix k has n_s_vec[k] rows and p columns.

Sigma0 True target covariance matrix, of dimension p by p.

SigmaS List of K true source covariance matrices.

R0 True target correlation matrix.

RS List of K true source correlation matrices.

var0 Length-p vector of true target marginal variances.

varS List of K length-p source variance vectors.

Examples

```
K <- 5
dat <- data.generate(p = 100, K = K, k0 = 5, s0 = 10,
                     n0 = 150, n_s_vec = rep(500, K), seed = 7)
X0 = dat$X0
XS = dat$XS
Sigma0 = dat$Sigma0
R0 = dat$R0
```

`fit.ETL.corr` *Entry-wise transfer learning for correlation matrix estimation*

Description

This function is used to implement ETL_corr method.

Usage

```
fit.ETL.corr(X0, XS, c_A1, c_A2, M, c_H = NULL, ncores, cl = NULL)
```

Arguments

`X0`	Target observations, with rows representing observations and columns representing variables.
`XS`	List of source datasets with the same variables and column order as `X0`. Source sample sizes may differ.
`c_A1`	Quantile probability used to truncate absolute source residuals after standardization.
`c_A2`	Quantile probability used to truncate absolute standardized source predictors.
`M`	Compatibility bound on $\lvert(\rho_s - \rho_t)y_l\rvert$. Larger values admit more source observations.
`c_H`	Multiplier of the source bivariate KDE bandwidth matrix. `NULL` uses `c_H = 1`.
`ncores`	Number of workers when `cl = NULL`. Internally created clusters are stopped on exit.
`cl`	Optional existing parallel cluster, which takes precedence over `ncores`. The caller must stop a supplied cluster. Use `NULL` to let the function manage execution according to `ncores`.

Value

A list with two components:

R_hat Symmetric numeric $p \times p$ pairwise target correlation estimate with unit diagonal, in input column order.

H_min Minimum weighted predictor sum-of-squares denominator across variable pairs, used in sparse correlation thresholding.

See Also

`fit.ETL.Scor`, `fit.ETL.corr.cv`, `fit.ETL.cov`, `data.generate`

Examples

```
K <- 5
dat <- data.generate(p = 100, K = K, k0 = 5, s0 = 10,
                     n0 = 150, n_s_vec = rep(500, K), seed = 7)
fit <- fit.ETL.corr(dat$X0, dat$XS, M = 0.3)
fit$R_hat
fit$H_min
```

`fit.ETL.corr.cv` *Select the ETL Correlation Compatibility Bound by Cross-Validation*

Description

Select a common correlation compatibility bound by target-only cross-validation and return the full-data ETL correlation estimate for the selected bound.

Usage

```
fit.ETL.corr.cv(X0, XS, c_A1, c_A2, M_grid, c_H = NULL,
  ncores, cl = NULL, cv_folds = 5, seed = NULL)
```

Arguments

`X0`	Target observations, with rows representing observations and columns representing variables.
`XS`	List of source datasets with the same variables and column order as `X0`. Source sample sizes may differ.
`c_A1`	Quantile probability used to truncate absolute source residuals after standardization.
`c_A2`	Quantile probability used to truncate absolute standardized source predictors.
`M_grid`	Candidate compatibility bounds for cross-validation. `NULL` uses `c(0.3, 0.6, 0.9)`. Supplied values replace this grid and are sorted and deduplicated. An exact loss tie selects the smallest bound.
`c_H`	Multiplier of the source bivariate KDE bandwidth matrix. `NULL` uses `c_H = 1`.
`ncores`	Number of workers when `cl = NULL`. Internally created clusters are stopped on exit.
`cl`	Optional existing parallel cluster, which takes precedence over `ncores`. The caller must stop a supplied cluster. Use `NULL` to let the function manage execution according to `ncores`.
`cv_folds`	Number of target cross-validation folds. Source observations are not split.
`seed`	Random seed for allocating target observations to CV folds. A supplied seed restores the caller's random-number state on exit. The default `NULL` uses and advances the current random-number stream.

Value

A list with eight components:

R_hat Symmetric numeric $p \times p$ full-data target correlation estimate at `M_selected`, with unit diagonal and input column order.

H_min Positive scalar equal to the minimum of `H_pair`. This is a thresholding denominator, not a KDE bandwidth.

rho_pair Numeric vector of $p(p-1)/2$ full-data pairwise correlations at `M_selected`, ordered according to `pair_id`.

H_pair Positive numeric vector of weighted predictor sum-of-squares denominators for the full-data selected fit, in pair order. These values are not sample counts.

pair_id Integer two-column matrix of lower-triangle indices (j, l) in R's column-major lower-triangle order.

M_selected Scalar compatibility bound chosen by target CV.

M_grid Sorted unique candidate bounds actually evaluated, including the resolved grid when the input is `NULL`.

cv_loss Named numeric vector of average validation squared prediction losses, aligned with `M_grid`; smaller is better.

See Also

`fit.ETL.corr`, `fit.ETL.Scor`, `data.generate`

Examples

```
K <- 5
dat <- data.generate(p = 100, K = K, k0 = 5, s0 = 10,
                     n0 = 150, n_s_vec = rep(500, K), seed = 7)
fit <- fit.ETL.corr.cv(dat$X0, dat$XS, M_grid = NULL,
                       cv_folds = 5, seed = NULL)
fit$M_grid
fit$M_selected
fit$cv_loss
custom <- fit.ETL.corr.cv(dat$X0, dat$XS, M_grid = c(0.2, 0.4),
                          cv_folds = 5, seed = NULL)
custom$M_selected
```

`fit.ETL.Scor` *ETL-Scor: Compute a Sparse ETL Correlation Estimate*

Description

Apply a hard threshold to an ETL correlation estimate.

Usage

```
fit.ETL.Scor(R_hat, H_min, c_T = NULL)
```

Arguments

`R_hat`	ETL correlation estimate returned as `R_hat` by `fit.ETL.corr` or `fit.ETL.corr.cv`.
`H_min`	Value supplied by the `H_min` component of the output from `fit.ETL.corr`.
`c_T`	Hard-threshold multiplier. Larger values remove more off-diagonal correlations; zero retains all off-diagonal estimates. `NULL` uses 1.

Details

For a matrix with p variables, the threshold is

$$c_T \sqrt{\log(p)/H_{\min}}.$$

An upper-triangle entry is set to zero only when its absolute value is strictly less than this threshold. Equality is retained. The upper triangle is copied to the lower triangle and the diagonal is set to one.

This function is directly callable after a correlation fit and does not fit marginal variances or require the original observations. Thresholding does not guarantee positive semidefiniteness, and retained entries are not clipped to [-1,1]. Use `fit.ETL.Scov` to combine this `R_hat` component with marginal variances and obtain the adjusted sparse covariance.

Value

A list with two components:

R_hat Estimated sparse correlation matrix, with unit diagonal and hard-thresholded off-diagonal entries, in the input variable order. Input dimension names are retained.

threshold The threshold used, equal to $c_T \sqrt{\log(p)/H_{\min}}$ after resolving `c_T = NULL` to 1.

See Also

`fit.ETL.corr`, `fit.ETL.corr.cv`, `fit.ETL.Scov`

Examples

```
K <- 5
dat <- data.generate(p = 100, K = K, k0 = 5, s0 = 10,
                     n0 = 150, n_s_vec = rep(500, K), seed = 7)
fit <- fit.ETL.corr(dat$X0, dat$XS)
fit_scor <- fit.ETL.Scor(R_hat = fit$R_hat, H_min = fit$H_min,
                         c_T = NULL)
fit_scor$R_hat
fit_scor$threshold
```

`ETL.var` *Coordinate-wise transfer learning for variance estimation*

Description

Estimate target marginal variances using supplied truncation and variance-compatibility parameters.

Usage

```
ETL.var(X0, XS, A, M_var, ncores, cl = NULL)
```

Arguments

X0	Target data matrix or data frame, with observations in rows and variables in columns.
XS	List of source data matrices or data frames with the same variables and column order as X0.
A	Bound for truncating centered source residuals, in the original variable units. A residual u is retained when $\|u\| \leq A$.
M_var	Source variance-compatibility tolerance. A source contributes when $\|v_k/v_0 - 1\| \leq M_{\text{var}}$ for that coordinate.
ncores	Number of workers when cl = NULL. Internally created clusters are closed after use.
cl	Optional existing **parallel** cluster. NULL lets the function manage execution according to ncores. The caller closes an externally supplied cluster.

Value

A list with three components:

ETL_var_hat Estimated target marginal variances in input column order.

A The supplied source-residual truncation bound.

M_var The supplied variance-compatibility tolerance.

See Also

ETL.var.cv, fit.ETL.cov, fit.ETL.Scov

Examples

```
K <- 5
dat <- data.generate(p = 100, K = K, k0 = 5, s0 = 10,
                     n0 = 150, n_s_vec = rep(500, K), seed = 7)
fit <- ETL.var(dat$X0, dat$XS, A = 3, M_var = 0.4)
fit$ETL_var_hat
```

ETL.var.cv *Coordinate-wise transfer learning for variance estimation by cross-validation*

Description

Estimate target marginal variances by screened, density-ratio-weighted transfer from source samples. Choose a common pair of truncation and variance-compatibility tuning parameters using target-only cross-validation.

Usage

```
ETL.var.cv(X0, XS, A_grid, M_var_grid, nfold = 5, ncores,
  seed = NULL, cl = NULL)
```

Arguments

`X0`	Target data matrix or data frame, with observations in rows and variables in columns. The variables must appear in the same order as in every source in `XS`.
`XS`	List of source data matrices or data frames, with observations in rows and variables in columns. Each source uses the same variables and column order as `X0`; sample sizes may differ across domains.
`A_grid`	Candidate bounds for truncating source residuals. `NULL` uses `c(2.5, 3, 3.5, 4)`; supplying a grid replaces those candidates. A source residual $u = x - \bar{x}$ is retained when $\lvert u\rvert \leq A$. Bounds are in the original variable units, not standardized units or quantile probabilities. Values are sorted and duplicates removed.
`M_var_grid`	Candidate tolerances for source variance compatibility. `NULL` uses `c(0.2, 0.4, 0.6, 0.8, 1)`; supplying a vector replaces that grid independently of `A_grid`. A source contributes to a coordinate when $\lvert v_k/v_0 - 1\rvert \leq M_{\mathrm{var}}$, where v_k and v_0 are that coordinate's source and target sample variances. During cross-validation, v_0 uses only target training observations; the final fit uses all target observations. Values are sorted and duplicates removed. This variance-ratio tolerance is distinct from the correlation estimator's tuning parameter.
`nfold`	Number of folds used to select the variance tuning parameters by target-data cross-validation.
`ncores`	Number of worker processes requested when `cl = NULL`. Use one for sequential execution; internally created clusters are closed after use. An external `cl` determines its own worker count.
`seed`	Seed controlling the balanced, shuffled allocation of target observations to folds. A supplied seed preserves the caller's random state; `NULL` uses and advances the current random stream.
`cl`	Existing **parallel** cluster used for computation across coordinates. With `NULL`, execution is controlled by `ncores`. The caller is responsible for closing an externally supplied cluster.

Value

A list with seven components:

ETL_var_hat Positive finite numeric vector of length p, the estimated target marginal variances in input column order. These are variances, not standard deviations.

A_hat Scalar truncation bound selected by global target CV.

M_var_hat Scalar variance-ratio tolerance selected by global target CV.

cv_loss Numeric matrix of mean CV losses. Rows correspond to `A_grid`; columns correspond to `M_var_grid`. Row labels begin with `A=`; column labels begin with `M_var=`. Smaller is better. A nonfinite candidate has loss `Inf`.

A_grid Sorted unique truncation bounds actually evaluated, including the resolved default grid when the input is `NULL`.

M_var_grid Sorted unique variance-ratio tolerances actually evaluated, including the resolved default grid when the input is `NULL`.

cv_loss_coordinate Numeric matrix with p rows and `length(A_grid) * length(M_var_grid)` columns. Each row is one coordinate; column order is `expand.grid(A = A_grid, M_var = M_var_grid)`, with `A` varying fastest. Column means produce the corresponding global losses.

See Also

fit.ETL.corr, make.pd

Examples

```
K <- 5
dat <- data.generate(p = 100, K = K, k0 = 5, s0 = 10,
                     n0 = 150, n_s_vec = rep(500, K), seed = 7)
fit <- ETL.var.cv(dat$X0, dat$XS, A_grid = NULL,
                  M_var_grid = NULL, nfold = 5, seed = NULL)
fit$ETL_var_hat
fit$A_hat
fit$M_var_hat
fit$cv_loss
```

---

make.pd *Positive-Definite Adjustment by a Diagonal Shift*

Description

Symmetrize a square matrix and, when needed, add a nonnegative constant to every diagonal entry. This is the supplied positive-definite adjustment used in ETL-cov and ETL-Scov.

Usage

```
make.pd(S, min_eig)
```

Arguments

S	Square matrix to be symmetrized and adjusted to positive definiteness by adding a constant to its diagonal.
min_eig	Desired lower bound for the smallest eigenvalue of the adjusted matrix, in the same units as S.

Value

A symmetric numeric matrix with the same dimensions as S, with smallest eigenvalue at least min_eig, up to floating-point precision. The return value is a matrix, not a list.

See Also

fit.ETL.cov, fit.ETL.Scov

Examples

```
S <- matrix(c(1, 1.2, 1.2, 1), 2, 2)
S_pd <- make.pd(S, min_eig = 0.1)
eigen(S_pd, symmetric = TRUE, only.values = TRUE)$values
```

`fit.ETL.cov` *ETL-cov: Covariance Estimation from Correlation and Variance Estimates*

Description

Combine entry-wise correlation estimates and coordinate-wise variance estimates using the correlation-variance decomposition, then apply the supplied positive-definite adjustment.

Usage

```
fit.ETL.cov(ETL_corr_hat, ETL_var_hat, min_eig)
```

Arguments

`ETL_corr_hat`	Target correlation estimate, normally the `R_hat` component from `fit.ETL.corr` or `fit.ETL.corr.cv`. Its rows and columns must follow the same variable order as `ETL_var_hat`.
`ETL_var_hat`	Target marginal variance estimates, normally the `ETL_var_hat` component returned by `ETL.var` or `ETL.var.cv`, in the same variable order as `ETL_corr_hat`. Their square roots rescale the correlation estimate into covariance units.
`min_eig`	Desired lower bound for the smallest eigenvalue of the final covariance matrix, enforced by a diagonal adjustment.

Value

The ETL-cov estimate: a symmetric positive-definite numeric $p \times p$ covariance matrix in input variable order. It is a matrix, not a list. The eigenvalue bound is subject to floating-point precision.

See Also

`fit.ETL.corr`, `ETL.var`, `make.pd`, `fit.ETL.Scov`

Examples

```
K <- 5
dat <- data.generate(p = 100, K = K, k0 = 5, s0 = 10,
                     n0 = 150, n_s_vec = rep(500, K), seed = 7)
cor_fit <- fit.ETL.corr(dat$X0, dat$XS)
var_fit <- ETL.var(dat$X0, dat$XS, A = 3, M_var = 0.4)
ETL_cov_hat <- fit.ETL.cov(cor_fit$R_hat, var_fit$ETL_var_hat)
ETL_cov_hat
```

`fit.ETL.Scov` *ETL-Scov: Sparse Covariance Estimation*

Description

Combine sparse correlation estimates and coordinate-wise variance estimates, then apply the supplied positive-definite adjustment.

Usage

```
fit.ETL.Scov(ETL_scor_hat, ETL_var_hat, min_eig)
```

Arguments

`ETL_scor_hat`	Sparse target correlation estimate, normally the `R_hat` component returned by `fit.ETL.Scor`. Its rows and columns must follow the same variable order as `ETL_var_hat`.
`ETL_var_hat`	Target marginal variance estimates, normally the `ETL_var_hat` component returned by `ETL.var` or `ETL.var.cv`, in the same variable order as `ETL_scor_hat`. Their square roots rescale the sparse correlation estimate into covariance units.
`min_eig`	Desired lower bound for the smallest eigenvalue of the final sparse covariance matrix, enforced by a diagonal adjustment.

Value

The ETL-Scov estimate: a symmetric positive-definite numeric $p \times p$ covariance matrix with the sparse input's off-diagonal zero pattern.

See Also

`fit.ETL.Scor`, `ETL.var`, `make.pd`, `fit.ETL.cov`

Examples

```
K <- 5
dat <- data.generate(p = 100, K = K, k0 = 5, s0 = 10,
                     n0 = 150, n_s_vec = rep(500, K), seed = 7)
cor_fit <- fit.ETL.corr(dat$X0, dat$XS)
scor_fit <- fit.ETL.Scor(cor_fit$R_hat, cor_fit$H_min, c_T = NULL)
ETL_scor_hat <- scor_fit$R_hat
var_fit <- ETL.var(dat$X0, dat$XS, A = 3, M_var = 0.4)
ETL_scov_hat <- fit.ETL.Scov(ETL_scor_hat, var_fit$ETL_var_hat)
ETL_scov_hat
```

`fit.domain.scor` *Compute a Sparse Single-Domain Initial Estimate*

Description

Threshold a single domain's sample covariance matrix and optionally convert the result to a correlation matrix. The correlation estimators use this function to obtain their initial within-domain correlations.

Usage

```
fit.domain.scor(X, corr)
```

Arguments

`X`	Single-domain observations, with rows representing observations and columns representing variables.
`corr`	Whether to return correlations: `TRUE` converts the thresholded covariance to a correlation matrix, while `FALSE` returns the thresholded covariance matrix.

Value

A symmetric numeric $p \times p$ matrix in the input column order. With `corr = TRUE`, it is a sparse correlation matrix with unit diagonal. With `corr = FALSE`, it is a sparse covariance matrix whose diagonal equals the sample variances. The return value is a matrix, not a list. Input column names, when present, label both dimensions.

See Also

`fit.ETL.corr`, `fit.ETL.corr.cv`, `make.pd`

Examples

```
K <- 5
dat <- data.generate(p = 100, K = K, k0 = 5, s0 = 10,
                     n0 = 150, n_s_vec = rep(500, K), seed = 7)
R_initial <- fit.domain.scor(dat$X0, corr = TRUE)
S_initial <- fit.domain.scor(dat$X0, corr = FALSE)
R_initial
diag(S_initial)
```

Index

data.generate, 1, *3*, *5*

ETL.var, 6, *10, 11*
ETL.var.cv, *7*, 7

fit.domain.scor, 12
fit.ETL.corr, 3, *5, 6*, *10*, *12*
fit.ETL.corr.cv, *3*, 4, *5, 6*, *12*
fit.ETL.cov, *3*, *7*, *9*, 10, *11*
fit.ETL.Scor, *3*, *5*, 5, *11*
fit.ETL.Scov, *6, 7*, *9, 10*, 11

make.pd, 9, *10–12*